Pennine Pizzeria Recommendation

COM4401 · SOFTWARE ENGINEERING FUNDAMENTALS

SOLUTION THROUGH WATERWALL METHODOLOGY

CONTENTS

1. Analysis 2

1.1 Customer Data sheet 2

1.2 Requirements Table 2

1.3 IPO Table (Input – Process – Output) 3

1.4 Assumptions and questions for the client 4

1.5 Validation Rules 4

2. Design 5

2.1 Explanation for way of working 5

2.2 Subroutine list 5

2.3 Data plan 7

2.4 Main flow sketch 8

2.5 Requirements check 9

2.6 Pseudocode 11

2.7 test table 16

3. Build 18

3.1 Instructions on running the program 18

3.2 Program flowcharts 18

3.2.1 Main 19

3.2.2 getCustomerDetails 20

3.2.3 checkBlank 21

3.2.4 getNumPizzas 22

3.2.5 pizzaDetails 23

3.2.6 setSizeOfPizzas ..... **Error! Bookmark not defined.**

3.2.7 setToppings 25

3.2.8 checkDelivery 26

3.2.9 displayBill 27

3.3 How the build matches with the design 28

3.4 Programming conventions followed 28

3.5 Sample Run 29

4. Testing 30

1. Analysis

1.1 Customer Data sheet

This is the data that the staff will be imputing into the program

Item	Rule
Minimum number of pizzas	1
Maximum number of pizzas	6
Small pizza	£3.25
Medium pizza	£5.50
Large pizza	£7.15
1 extra topping	£0.75
2 extra toppings	£1.35
3 extra toppings	£2.00
4 or more extra toppings	£2.50
Discount	10% discount if the order subtotal is over £20 before delivery is applied
Delivery charge	£2.50 if delivery is requested

1.2 Requirements Table

Requirements

ID	Requirements	Type
R1	Staff can enter Customer name, Address and phone number	Input
R2	Staff can enter the Amount of pizzas ordered	Input
R3	Staff can enter the Size of each pizza ordered	Input
R4	Staff can enter the number of extra toppings for each pizza	Input
R5	Whether Delivery is required	Input
R6	Whether Discount is required	Input
R7	Output a bill showing the Customer Details	Output
R8	Output the total number of pizzas chosen and price per pizza	Output

R9	Output a full Total of standard cost of pizza with extra toppings	Process
R10	Order Subtotal before discount and delivery	Input
R11	Putting Any discount applied	Input
R12	Any delivery charge	Process
R13	Final total cost of order	Output
R14	The program is user friendly and generally easy to use, with easy prompts and concise yet productive error messages.	Non-functional
R15	The program works fully and does not let the user input invalid data	Non-functional
R16	The program is efficient and made to run well, following standard programming conventions	Non-functional

Customer name, address and phone number

The number of pizzas ordered

The size of each pizza

The number of extra toppings for each pizza

Whether delivery is required

Whether discount is required

Output a bill showing the customer details

Output the total number of pizzas and price per pizza

Output a total number of pizzas chosen and price per pizza

Output a full total of standard cost of pizza with extra toppings

Order subtotal before Discount and delivery

Putting any discount applied

Putting any delivery charge

1.3 IPO Table (Input – Process – Output)

Input	Process	Output
Customer name, address, phone(R1)	Check each is not blank	Customers details on the bill(R6)
Number of pizzas ordered(R2)	Repeat the input from 1 to 6	Number of pizzas ordered(R7)

Size of each pizza ordered(R3)	Repeat the input from 1 to 3	Size of the pizzas ordered(R8)
Number of extra toppings per pizza(R4)	Repeat the input from 1 to 5	Number of toppings ordered(R8)
Delivery requirement(R5)	Check if required	Addition of delivery charge to the order total + £2.50
Discount Application(R6)	Check if applied	Reduction of the order total by 10% if order total >£20 excluding delivery
	Order Total= total of pizzas ordered + topping total+ delivery(if required)- discount(if applicable)	Total cost R10-R13

1.4 Assumptions and questions for the client

Does not Digress...	Decision
How the data staff input should be stored.	Customer info will be stored as string; Delivery will be a Boolean, and the rest will be Integers.
Weather Vat has already applied.	Treated as though Vat is already applied

1.5 Validation Rules

Name,address,phone	Not blank: the first character must not be a space, and an empty entry is rejected	“This field cannot be left blank, please try again”
Number of pizzas	Whole number from 1 to 6	“Error,orders should be between 1 to 5 pizzas”
Number of topings	Whole number from 1 to 5	“Error,please add between 1 to 5 toppings”

2. Design

2.1 Explanation for way of working

Based on Pandeli and Wayne's analysis of the clients brief and requirements, I considered their requirements table and IPO table and used some analytical skills of my own to use the data and information they gathered to create 3 designs that will help aid Henry create the Flowgarithm and final algorithm that will run the program to ensure that it works.

I will first need to decompose the analysis so that it is easy to understand and make it easier to create a functional and correct design that will help Henry create the program. I will make a subroutine list, a data plan, an array list and what each array holds inside it so that the brief is fully decomposed and simplified so that the design can be made to be easily understood

I will make a simple flowchart that should be easy to understand, this will help the client understand how the actual program is meant to run and will run, and it is important to keep the flowchart simple so that the clients aren't confused since they most likely won't be well versed in pseudo code and actual programming language and syntax.

I will make a relatively simple pseudo code that will be easy and simple to understand for a programmer so that Henry will be able to interpret it easily and port it over onto the actual Flowgarithm software that will run the program. The pseudo code won't be the actual final code and will only be a base template that will help Henry understand what the code is meant to look and function like, and it isn't meant to be a replacement for the actual functioning code.

I will then make a test plan design, so that when the final code is created, it can be tested and probed by Haneef to ensure that the program functions as intended and is able to run without any logic errors, syntax errors or runtime and performance errors. Creating this table will allow Haneef to easily do all these tests and create tests logs with the expected outcome, actual outcome, and other possible notes that could be made

2.2 Subroutine list

This is the list of subroutines that the program will need to run and function and having a list of all the subroutines allows us to fully decompose and understand the clients brief and requirements so that when we create the designs and final program, we haven't missed any details that were mentioned in the design brief

Name	Type	Takes in	Gives back / does
main( )	Module	Nothing	Sets up the variables and arrays, then calls the other subroutines in order
getCustomerDetails(customerDetails[ ])	Module	customerDetails array (empty)	Asks for and fills in the name, address and phone number
getNumPizzas( )	Function (Integer)	Nothing	Returns the number of pizzas (1 to 6). Shows an error and asks again if invalid
pizzaDetails(numOfPizza, sizeOfPizzas[ ], numOfToppings[ ])	Module	Number of pizzas and the two empty arrays	Goes through each pizza, fills the arrays with its size cost and topping cost, and shows the cost of that pizza
setSizeOfPizzas(count)	Function (Real)	The pizza number (count)	Asks for the size (1, 2 or 3) until valid. Returns the standard cost of that size
setToppings(count)	Function (Real)	The pizza number (count)	Asks for the number of extra toppings until valid. Returns the topping cost

checkDelivery( )	Function (Boolean)	Nothing	Asks if the order is being delivered. Returns True or False
displayBill(numOfPizza, sizeOfPizza[ ], numOfToppings[ ], isDelivery)	Module	Number of pizzas, the two filled arrays and isDelivery	Adds up all the costs, takes off the 10% discount if over £20, adds £2.50 if delivered, and shows the discount and order total

2.3 Data plan

Arrays are the method of storing data we have been told to use so I created a data plan, so that we can understand all the data that the program will be using, what type of data it is and what information is stored inside the data, this allows us to create an easy-to-follow design when developing the final program.

To explain, arrays start counting from 0, so PizzaSize[0] is pizza 1 and PizzaSize[5] is pizza 6. there are only 6 cells because the max number of pizzas that can be ordered it 6

Single values

Name	Type	Holds	Set by	Used by
numOfPizza	Integer	How many pizzas are in the order (1 to 6)	getNumPizzas	pizzaDetails, displayBill
isDelivery	Boolean	True if the order is being delivered	checkDelivery	displayBill

Arrays

Array	Type	What each cell holds	Written by	Used by
customerDetails[3]	String	One cell per detail (see below)	getCustomerDetails	Nothing yet

sizeOfPizzas[6]	Real	Standard cost of each pizza's chosen size (3.25, 5.50 or 7.15)	pizzaDetails	displayBill
numOfToppings[6]	Real	Extra topping cost of each pizza	pizzaDetails	displayBill

customerDetails[3]	Holds
[0]	Customer name
[1]	Customer address
[2]	Phone number

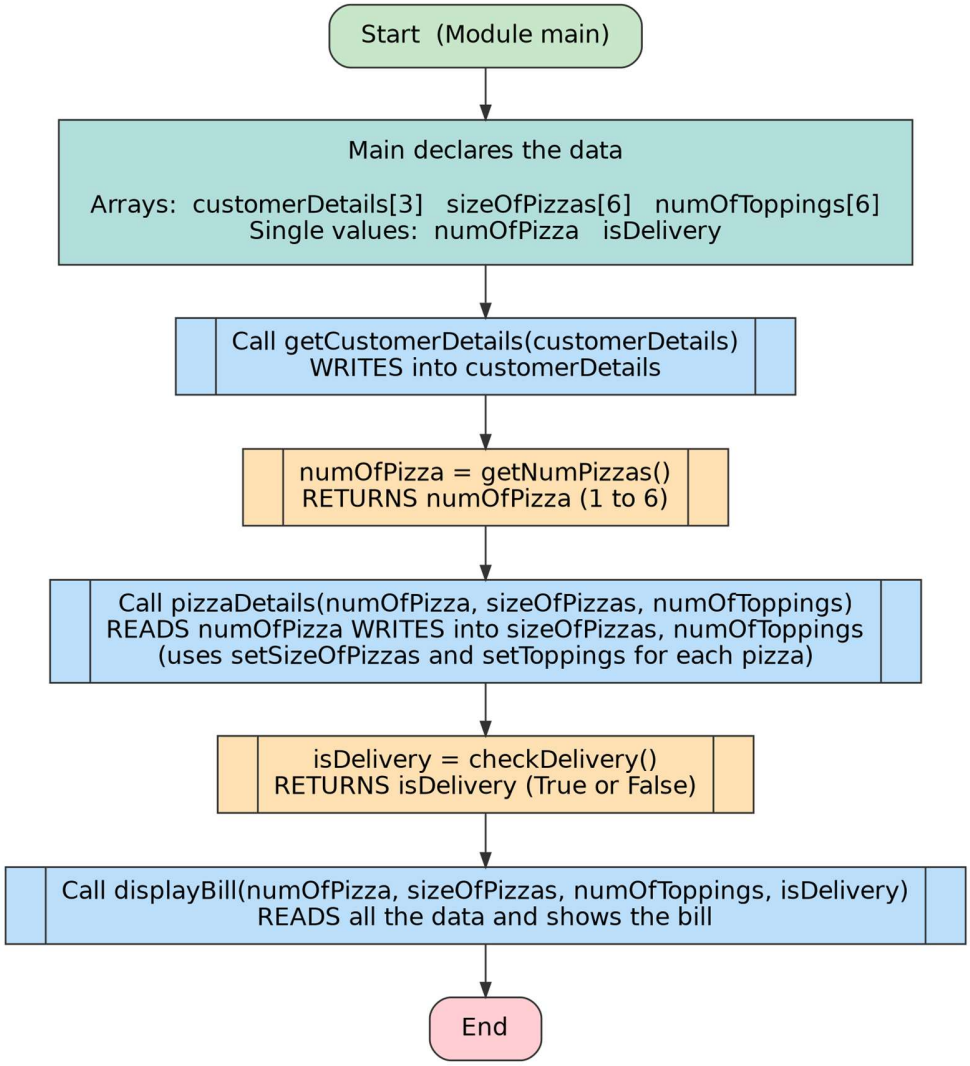
Working variables inside the subroutines

Subroutine	Working variables	Used for
getCustomerDetails	assign[3], i	Labels used in the prompts, and a loop counter
getNumPizzas	noOfPizzas	The number typed in, checked until it is 1 to 6
pizzaDetails	count	Counts through the pizzas
setSizeOfPizzas	sizeVal, label, cost	The size chosen, its name, and its cost
setToppings	val, cost	The toppings chosen and their cost
checkDelivery	check	The delivery answer
displayBill	sum, preSum, i	Running total, the subtotal before discount, and a loop counter

2.4 Main flow sketch

I used draw.io to create a flow chart that would be easily understandable for the client. The client doesn't need to understand how the program works, just what information is being used and accessed in the subroutines

This flowchart clearly explains where and when each subroutine is used, why it's being called and what it does. I was struggling to export the flowchart into this word document, so I just screenshotted the flowchart instead.



2.5 Requirements check

This is just requirements check to see that all the requirements that we analysed before are met and if a subroutine was created for the requirement, this helps us make sure that we haven't missed or forgot any client requirements and details

Req.	What the program must do	Subroutine	Where the data is
What staff enter			
R1	Customer name, address and phone number	getCustomerDetails	customerDetails[3]
R2	Number of pizzas ordered	getNumPizzas	numOfPizza
R3	Size of each pizza	setSizeOfPizzas (called by pizzaDetails)	sizeOfPizzas[6]
R4	Extra toppings for each pizza	setToppings (called by pizzaDetails)	numOfToppings[6]
R5	Whether delivery is required	checkDelivery	isDelivery
What the bill must show			
R6	Customer details	displayBill	customerDetails[3]
R7	Cost of each pizza	pizzaDetails	sizeOfPizzas[6], numOfToppings[6]
R8	Standard pizza cost and extra topping cost	displayBill	sizeOfPizzas[6], numOfToppings[6]
R9	Order subtotal before discount and delivery	displayBill	preSum
R10	Any discount applied	displayBill	sum, preSum
R11	Any delivery charge	displayBill	sum
R12	Final total cost of the order	displayBill	sum
Business rules			
R13	Minimum 1 and maximum 6 pizzas	getNumPizzas	numOfPizza
R14	Small £3.25, medium £5.50, large £7.15	setSizeOfPizzas	sizeOfPizzas[6]
R15	Toppings: 1 = £0.75, 2 = £1.35, 3 = £2.00, 4 or more = £2.50	setToppings	numOfToppings[6]
R16	10% discount if subtotal is over £20 before delivery	displayBill	sum, preSum

R17	£2.50 delivery charge if delivery is requested	displayBill	sum

2.6 Pseudocode

This is the Gaddis pseudocode for the program that was generated automatically by FLowgarithm, when the program was finished, this would let programmers read the program easily and understand what is happening in the program and how it works, it would allow them to edit and make changes to the program and translate the pseudocode into different programming languages like Python, Java, C++, etc.

```
Module main()
  // Setup Variables
  Declare String customerDetails[3]
  Declare Integer numOfPizza
  Declare Real sizeOfPizzas[6]
  Declare Real numOfToppings[6]
  Declare Boolean isDelivery

  // One Vars are set, Call GetCustomerDetails Subroutine.
  Call getCustomerDetails(customerDetails)

  // This sets the variable of 'NumOfPizza' using the 'getNumPizzas' subroutine
  Set numOfPizza = getNumPizzas()

  // Subroutine that sets the Pizza Size and Toppings
  Call pizzaDetails(numOfPizza, sizeOfPizzas, numOfToppings)
  Set isDelivery = checkDelivery()

  // Display Bill
  Call displayBill(numOfPizza, sizeOfPizzas, numOfToppings, isDelivery)
End Module

Function Boolean checkDelivery ()
  Declare Boolean check

  Display "Is this order going to be delivered? True or false only please."
```

```
  Input check

  Return check
End Function

Module displayBill (Integer numOfPizza, Real sizeOfPizza[], Real numOfToppings[], Boolean isDelivery)
  Declare Real sum
  Declare Real preSum

  Set sum = 0.0

  // The Do Loop itterates through each pizza size and topping and adds it to the sum
  Declare Integer i

  Set i = 0
  Do
    Set sum = sum + sizeOfPizza[i]
    Set sum = sum + numOfToppings[i]
    Set i = i + 1
  While numOfPizza > i

  // Checks for discount if sum > 20 before delivery
  Set preSum = sum
  If sum > 20 Then
    Display "Discount has been added to order"
    Set sum = sum * 0.9
  Else
    Display "Discount not earned for order"
  End If
  Display "Discount Ammount = £", preSum - sum

  // Checks if delivery has been selected
  If isDelivery == True Then
    Set sum = sum + 2.5
  End If
  Display "Order Subtotal = £", sum
End Module

Module getCustomerDetails (String customerDetails[])
```

```
  // This subroutine sets the details as such:
  // CustomerDetails[0]= Name
  // CustomerDetails[1]= Address
  // CustomerDetails[2]= Phone Number
  Declare String assign[3]

  Set assign[0] = "Name"
  Set assign[1] = "Address"
  Set assign[2] = "Phone Num"

  // Settup a for loop that itterates through customer details gathering the data for each feild recursively
  Declare Integer i

  For i = 0 To 2
    Display "Enter ", assign[i], "  :"
    Input customerDetails[i]
  End For
End Module

Function Integer getNumPizzas ()
  Declare Integer noOfPizzas

  Set noOfPizzas = 0
  Display "Enter number of pizzas(max 6)"
  While noOfPizzas < 1 OR noOfPizzas > 6
    Input noOfPizzas
    If noOfPizzas < 1 OR noOfPizzas > 6 Then
      Display "Error, must order minimum 1 and maximum 6 pizzas"
    Else
      Display "Ok, you've selected ", noOfPizzas, " pizzas."
    End If
  End While

  Return noOfPizzas
End Function

Module pizzaDetails (Integer numOfPizza, Real sizeOfPizzas[], Real numOfToppings[])
  Declare Integer count
```

```
  Set count = 0

  // Iterative Do loop that goes though each pizza's details
  Do

    // Function that sets the size of pizza
    Set sizeOfPizzas[count] = setSizeOfPizzas(count)

    // Function that sets number of toppings
    Set numOfToppings[count] = setToppings(count)
    Display "The cost for pizza ", count + 1, " is: £", numOfToppings[count] + sizeOfPizzas[count]
    Set count = count + 1

    // keep running till count reaches the number of pizzas
  While count < numOfPizza
End Module

Function Real setSizeOfPizzas (Integer count)
  Declare Integer sizeVal
  Declare String label
  Declare Real cost

  Set cost = 0
  Display "Select a size pizza you would like for pizza no", count + 1
  Do
    Display "1 = Small 2 = Medium 3 = Large"
    Input sizeVal
    If sizeVal > 3 OR sizeVal < 1 Then
      Display "Error: size must be within range"
    Else
      If sizeVal == 1 Then
        Set cost = cost + 3.25
        Set label = "small"
      Else
        If sizeVal == 2 Then
          Set cost = cost + 5.5
          Set label = "medium"
        Else
          Set cost = cost + 7.15
```

```
            Set label = "large"
          End If
        End If
        Display "Ok for pizza ", count + 1, " the size is: ", label, "."
      End If
    While sizeVal > 3 OR sizeVal < 1

    Return cost
End Function

Function Real setToppings (Integer count)
    Declare Integer val
    Declare Real cost

    Display "Select how many extra toppings for pizza no", count + 1
    Do
      Input val
      If val < 0 Then
        Display "Error, You cant have negative toppings!"
      Else
        If val >= 4 Then
          Set cost = 2.5
        Else
          If val == 3 Then
            Set cost = 2.0
          Else
            If val == 2 Then
              Set cost = 1.35
            Else
              Set cost = 0.75
            End If
          End If
        End If
      End If
      Display "Ok you've selected ", val, " extra toppings"
    While val < 0

    Return cost
End Function
```

Since Henry will be creating the program in flowgarithm, I didn't use any indentation as I believe that would confuse him and wasn't necessary since flowgarithm already does all the programming and syntax in the background based on the code that was created in the flow chart, I believe that this pseudocode will help guide Henry in creating the final program and should be basic enough to understand and follow when creating the final

2.7 Test table plan

This is the template for the test table that will be used when the final program is created. We're only doing simple black box testing, so I only made 3 tables, one for normal testing, boundary testing and extreme testing

Test table

Normal data

Test no.	Description	Input data	Expected result	Actual result	P / F

Boundary data

Test no.	Description	Input data	Expected result	Actual result	P / F

Extreme data

Test no.	Description	Input data	Expected result	Actual result	P / F

3. Build

This implementation is to the specifications of the Pizzaria company as well as following our own internal design specifications. The design was built in Flowgorithm and saved as PizzaOrderHandler.fprg.

3.1 Instructions on running the program

These are step by step instructions for using the program

1. Open Flowgorithm on your PC and navigate at the top to: 'File' then click 'Open', then select the file titled 'PizzaOrderHandler.fprg'.

2. Press on the green arrow pointing right (play button) to run the program

3.2 Program flowcharts

Each figure shows one function of PizzaOrderHandler.fprg, shape by shape, including its comments

Key:

Comments – Dotted Line Boxes

Declare – Yellow box with black lines

Assign – Yellow Box

Output – Green Box

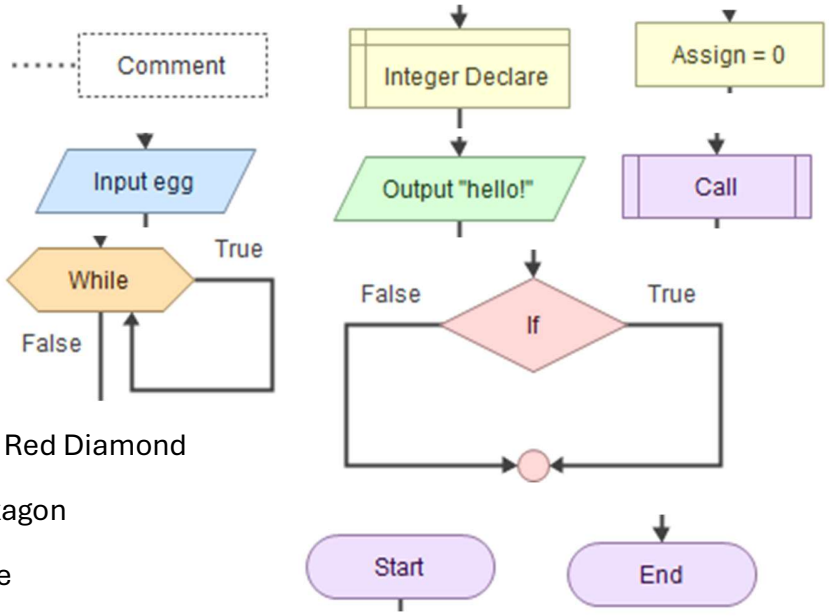
Input – Blue Box

Call(subroutine) – purple

'IF' Statement (true or false operator) - Red Diamond

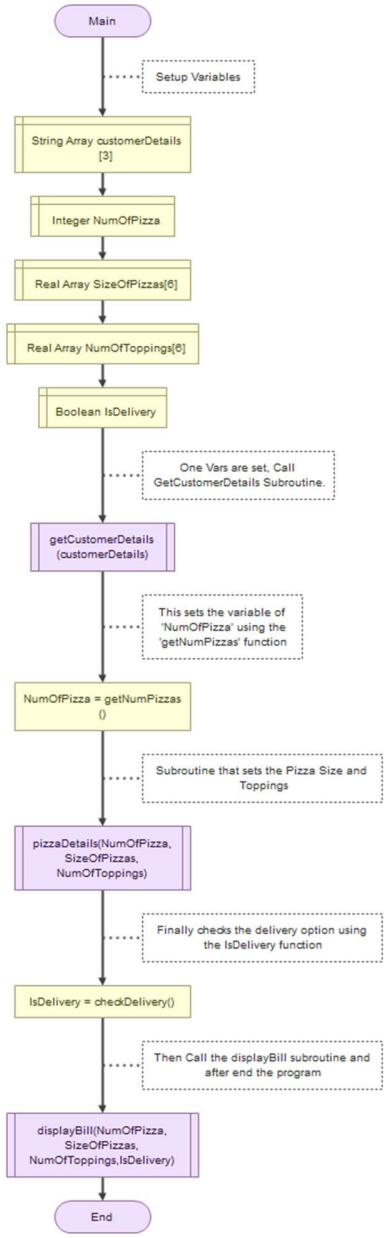
'While' 'Do' or 'For' loops – Orange Hexagon

Start/Return – purple rounded rectangle



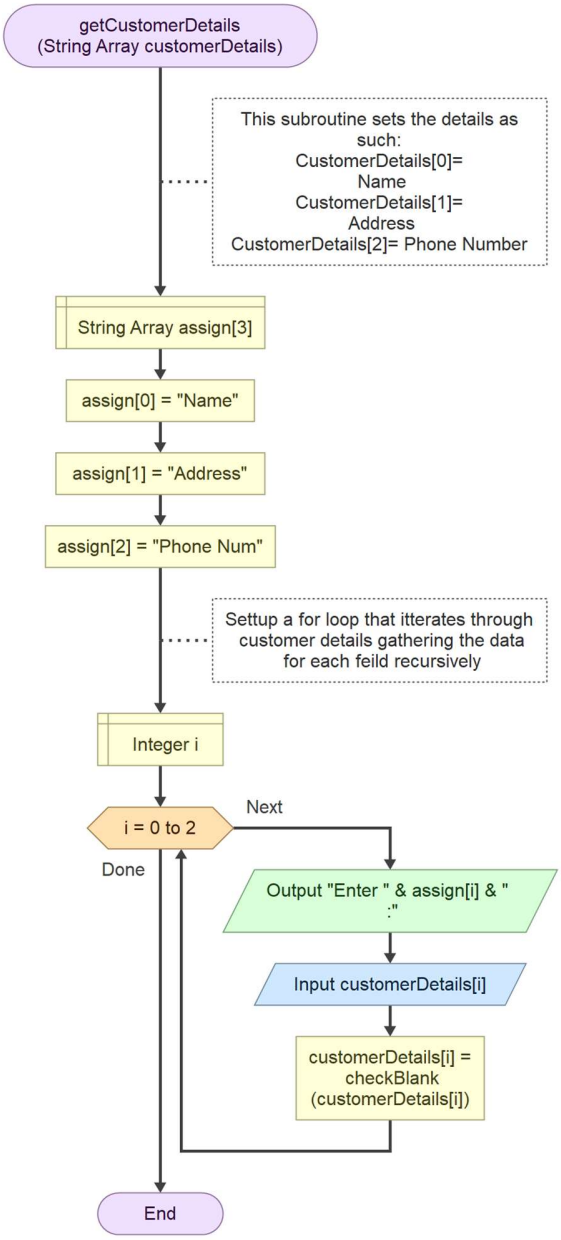
3.2.1 Main

Set up the variables and call each step in the ordering process sequentially, starting with getCustomerDetails:



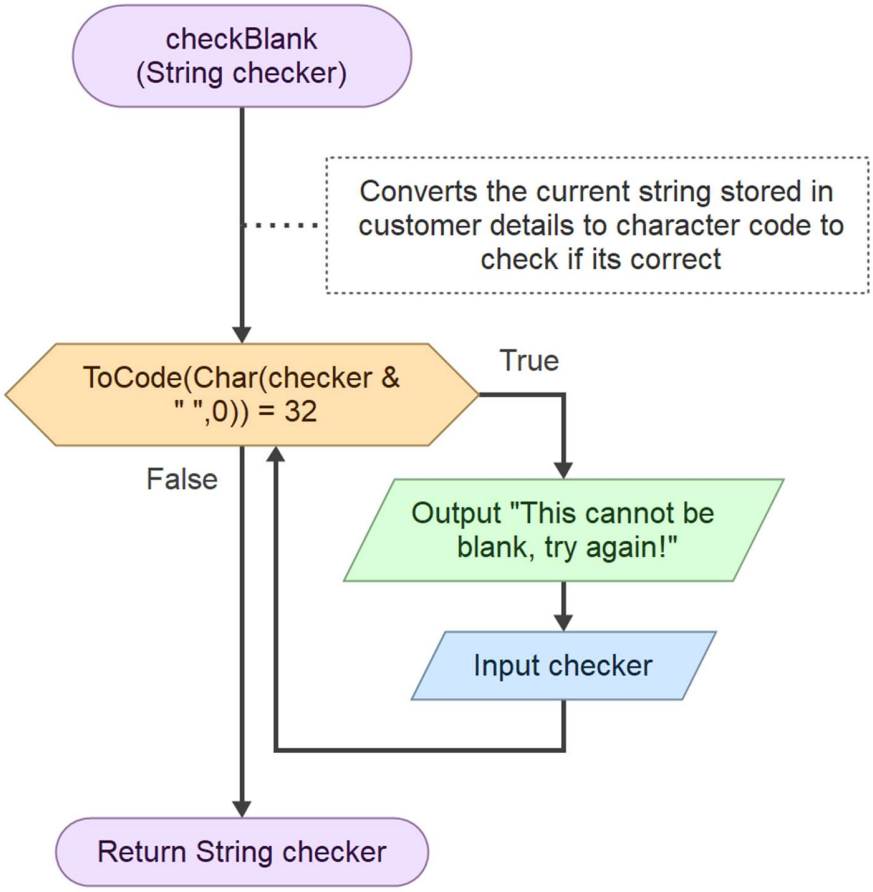
3.2.2 getCustomerDetails

This function Assigns the users inputs to the variable ‘customerDetails’ to be printed when the order is completed



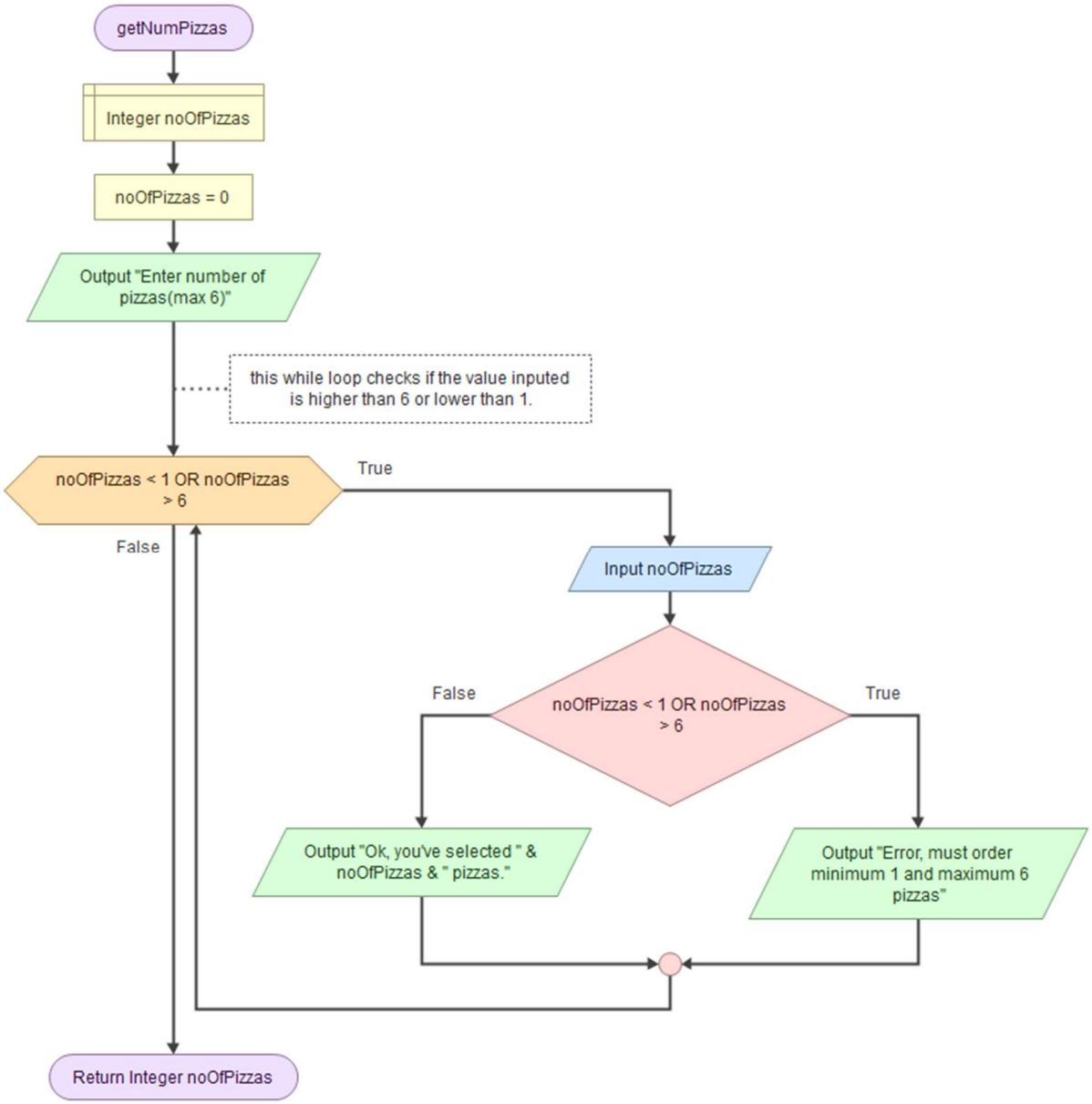
3.2.3 checkBlank

This is a simple function that uses an intrinsic function called ‘ToChar’ that converts the string to character code so that it can check to see if its value is blank, if so loop and tell the user to try again.



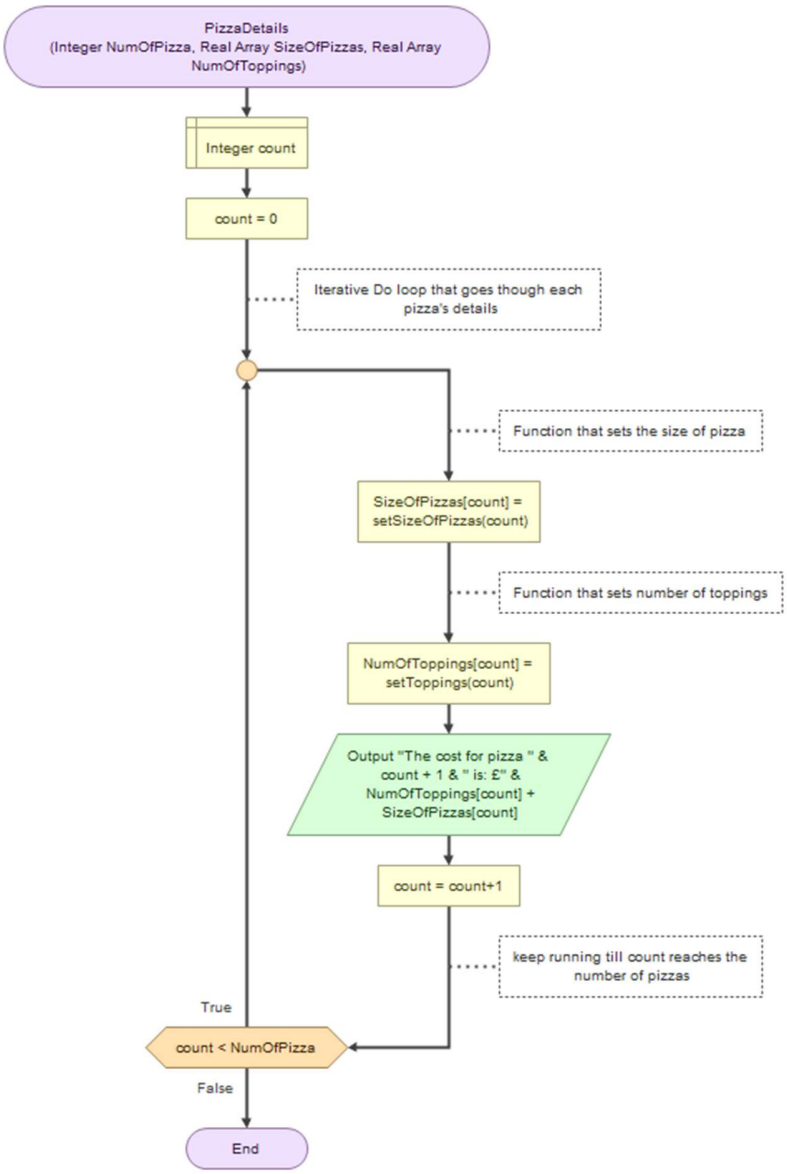
3.2.4 getNumPizzas

This function assigns the variable numPizza the amount of pizzas the customer has ordered up to 6. It checks if the value inputted is higher than 6 or lower than 0 before continuing.



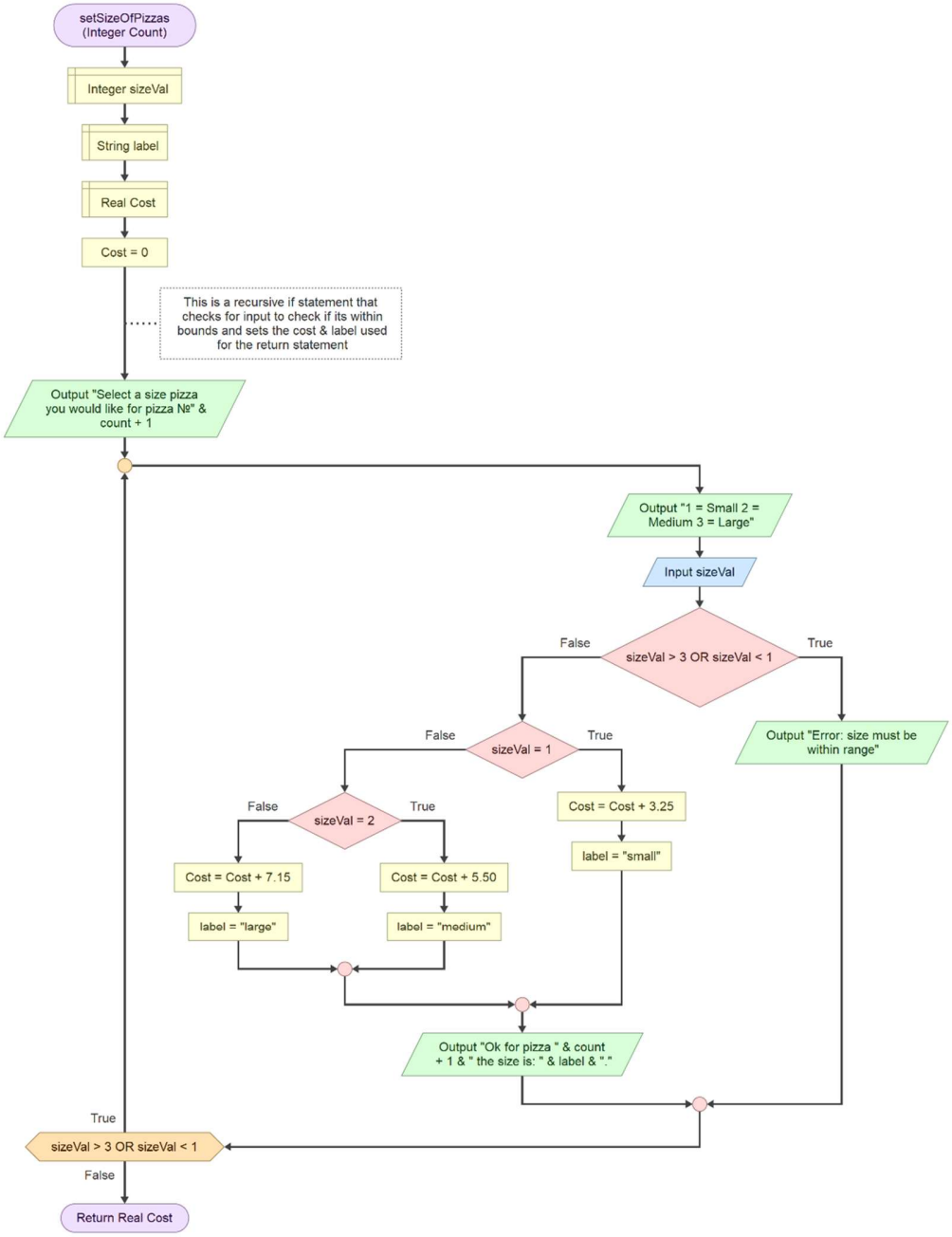
3.2.5 pizzaDetails

This subroutine purpose is to call two other functions (SizeOfPizzas & NumOfToppings) which set the size and amount of extra toppings if specified. It does this by incrementing a counter which runs in a loop until the 'DO' loop finds the counter is the same as the Number of pizzas ordered.



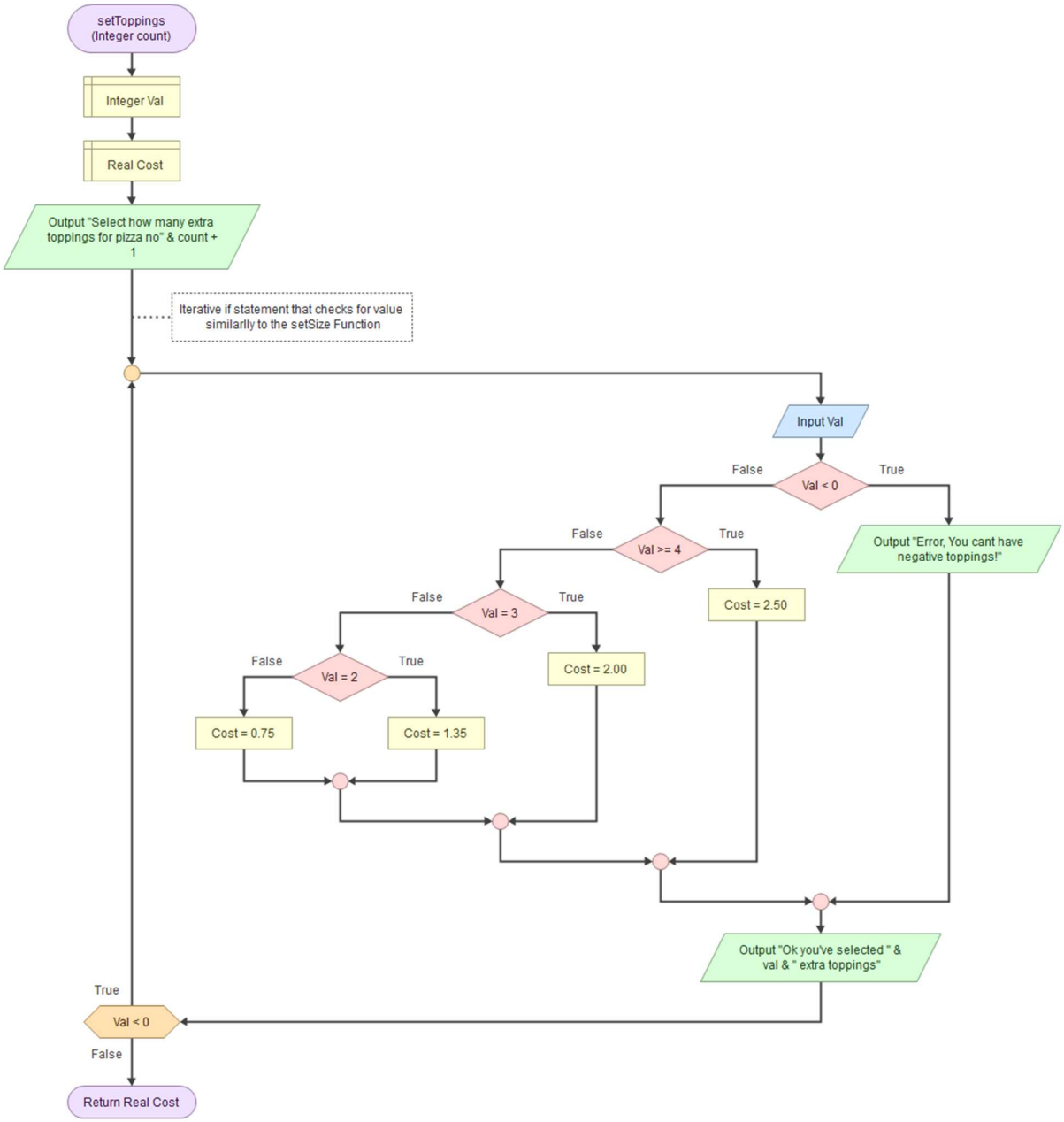
3.2.6 setSizeOfPizzas

This is a function that uses a nested IF statement that validated the inputted data and sets the variable array SizeOfPizzas to the cost of the size selected. It inputs the cost into the correct pizzas section by using the counter from 'PizzaDetails' as a reference point to the index in the array. This means it's reused for each pizza ordered.



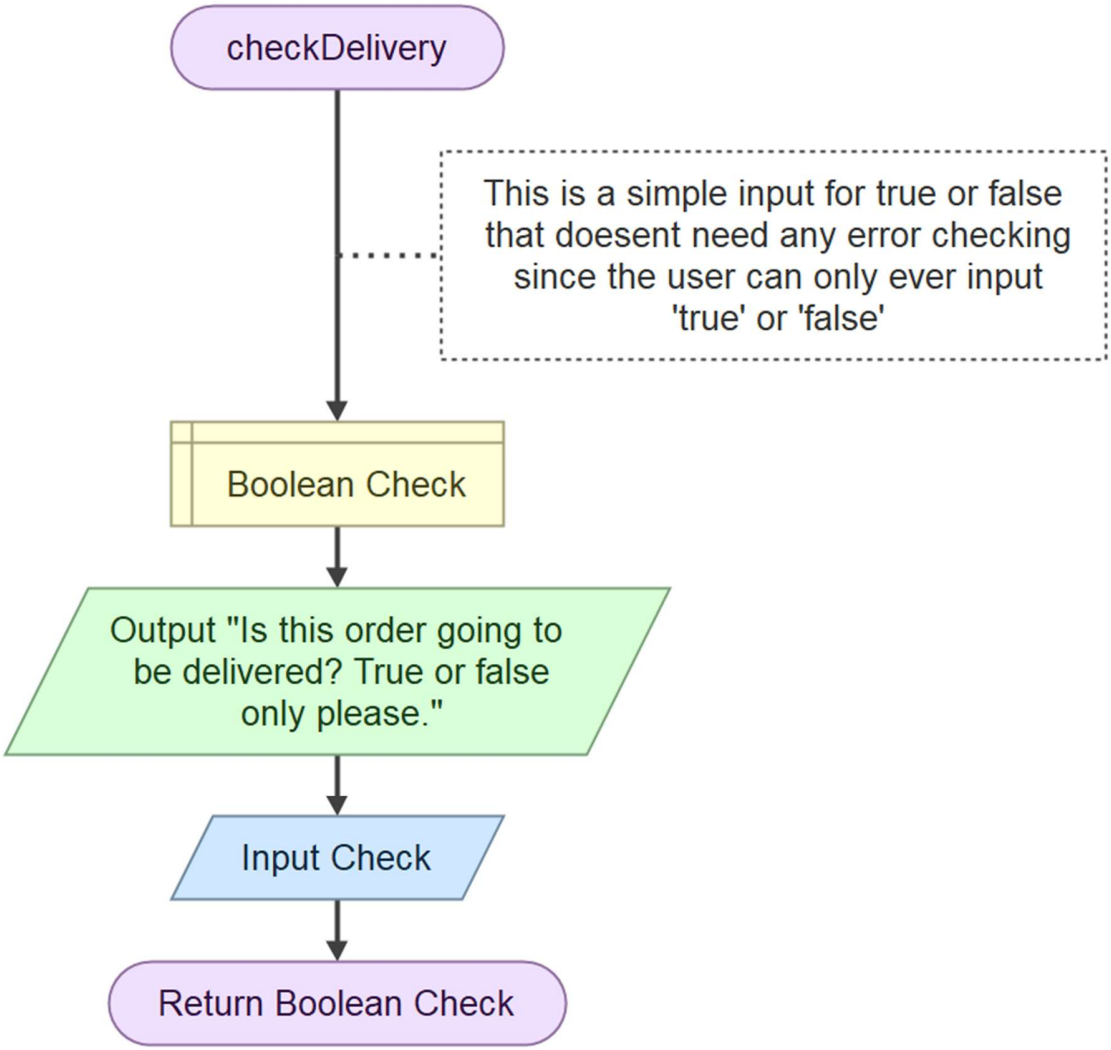
3.2.7 setToppings

This function's purpose is much the same as SetSizeOfPizzas just with the extra toppings instead, it's functionally the same but just with one more if statement and no upper limit to the amount of toppings, just the cost gets limited to £2.50.



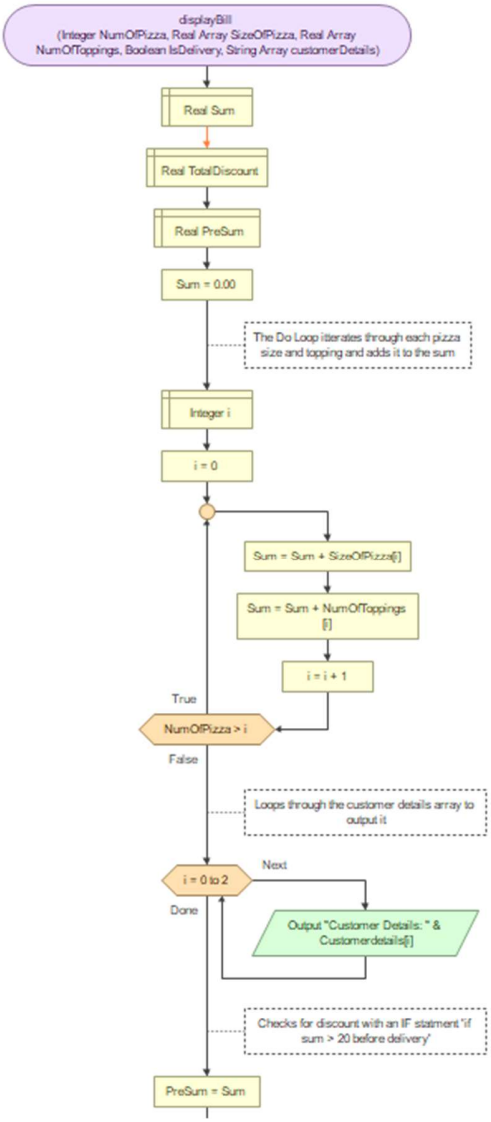
3.2.8 checkDelivery

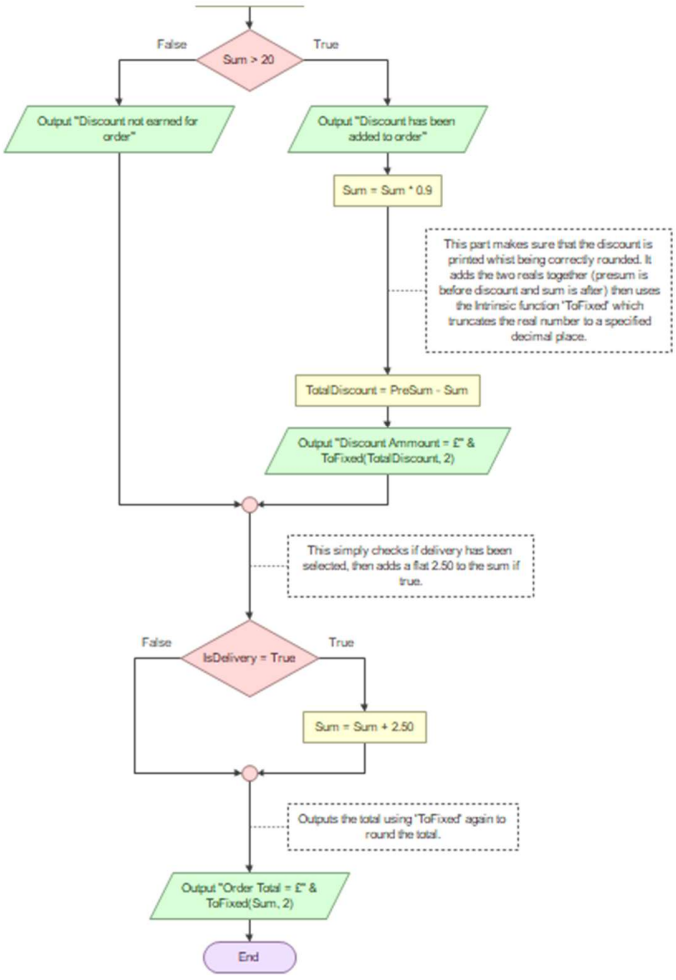
The penultimate and the simplest function of the whole program. All it does is ask for an input that is returned into a Boolean, no error checking necessary since inputs can't place exceptionally wrong data (like putting a string into an integer) so it's really quite simple. The only reason why this check is separate to the final 'displayBill' subroutine is that debugging is easier as well as making the code more human readable.



3.2.9 displayBill

This is the final subroutine and ultimate part of the program that recursively adds all the cost information from both arrays (SizeOfPizzas & NumOfToppings), Outputs the customer details, then checks if a discount is earned and calculates it and displays the amount saved if true, before finally checking the delivery and if it's true then adding a flat 2.50.





3.3 How the build matches with the design

Design	Implementation
Main() Flowchart design Subroutines	Main implementation is the same as the mockup Draw.io design 'to a tee'. Every function and subroutine is called at the same time and named the same as is specified in the mockup.
Arrays	customerDetails, SizeOfPizzas and NumOfToppings are all declared in main and follow the same naming convention as is specified in section 2.4
Data Validation Data accuracy	The function 'checkBlank' is exclusively made to ensure the data inputted into 'customerDetails' is not blank The Total cost and Savings amount is displayed using the ToFixed() intrinsic function that corrects the savings from being three decimal places (which doesn't make sense for currency) to two decimal places.

3.4 Programming conventions followed

- Comments – Labels that tell the other programmers and outside parties how the program works for less obfuscated code
- Accurate names – the variables subroutines and functions are all labeled appropriately and exactly as to ensure less confusion when reading or changing code e.g.: 'customerDetails' 'DisplayBill' or 'CheckDelivery'.
- Subroutine and function efficiency - One job for subroutine and usually multiple similar jobs for each function, so the code is cleaner.
- Syntax – No basic syntax or lexical errors; everything is defined correctly before usage.
- Variables – variables that aren't needed in multiple subroutines/functions are defined locally instead of globally.

Small improvements to be made: Could check the input for customerDetails to ensure the exact correct data is being inputted (i.e. phone number char length or no numbers in name)

3.5 Sample Run

This is the input and output from a typical order, Lines that are highlighted in yellow are inputs from staff members:

```
Enter Name  :
Genry
Enter Address  :
123 fake street
Enter Phone Num  :
0712345678
Enter number of pizzas(max 6)
2
Ok, you've selected 2 pizzas.
Select a size pizza you would like for pizza №1
1 = Small 2 = Medium 3 = Large
2
Ok for pizza 1 the size is: medium.
Select how many extra toppings for pizza no1
5
Ok you've selected 5 extra toppings
The cost for pizza 1 is: £8
Select a size pizza you would like for pizza №2
1 = Small 2 = Medium 3 = Large
3
Ok for pizza 2 the size is: large.
Select how many extra toppings for pizza no2
2
Ok you've selected 2 extra toppings
The cost for pizza 2 is: £8.5
Is this order going to be delivered? True or false only please.
true
Customer Details: Genry
Customer Details: 123 fake street
Customer Details: 0712345678
Discount not earned for order
Order Total = £19.00
```

4. Testing

4.1 Testing plan and results **Normal data**

Test no.	Description	Input data	Expected result	Actual result	P / F
1	One small pizza, one topping, collection	Pizzas: 1 Pizza 1: size 1, 1 topping Delivery: false	Pizza 1 £4.00. No discount. Order Total = £4.00	Pizza cost shown while ordering: £4. Discount not earned. Order Total = £4.00	Pass
2	One medium pizza, two toppings, with delivery	Pizzas: 1 Pizza 1: size 2, 2 toppings Delivery: true	Pizza 1 £6.85. No discount; delivery £2.50 added. Order Total = £9.35	Pizza cost shown while ordering: £6.85. Discount not earned. Order Total = £9.35	Pass*
3	Two different pizzas, no discount	Pizzas: 2 Pizza 1: size 3, 3 toppings Pizza 2: size 2, 1 topping Delivery: false	Pizza 1 £9.15; Pizza 2 £6.25. No discount. Order Total = £15.40	Pizza costs shown while ordering: £9.15, £6.25. Discount not earned. Order Total = £15.40	Pass
4	Discount earned (subtotal over £20)	Pizzas: 4 Every pizza: size 2, 1 topping Delivery: false	Each pizza £6.25. Discount applied (£2.50). Order Total = £22.50	Pizza costs shown while ordering: £6.25 each. Discount amount = £2.50. Order Total = £22.50	Pass
5	Discount and delivery together	Pizzas: 3 Every pizza: size 3, 2 toppings Delivery: true	Each pizza £8.50. Discount applied (£2.55); delivery £2.50 added. Order Total = £25.45	Pizza costs shown while ordering: £8.5 each. Discount amount = £2.55. Order Total = £25.45	Pass*
6	Bill shows every required item (use the order from test 5)	Same as test 5	Customer name, address and phone. Each pizza: standard £7.15 + toppings £1.35 = £8.50. Subtotal before discount and delivery £25.50. Discount £2.55. Delivery charge £2.50. Final total £25.45.	Customer details shown (name, address, phone). Pizza costs shown while ordering only (£8.5 each), not on the bill. Bill shows: discount amount £2.55 and Order Total £25.45.	Fail

				Missing from bill: standard and topping cost, subtotal before discount, delivery charge.	

Boundary data

Test no.	Description	Input data	Expected result	Actual result	P / F
7	Minimum pizzas (1)	Pizzas: 1 Pizza 1: size 3, 1 topping Delivery: false	Pizza 1 £7.90. No discount. Order Total = £7.90	Pizza cost shown while ordering: £7.9. Discount not earned. Order Total = £7.90	Pass
8	Maximum pizzas (6)	Pizzas: 6 Every pizza: size 1, 1 topping Delivery: false	Each pizza £4.00. Discount applied (£2.40). Order Total = £21.60	Pizza costs shown while ordering: £4 each. Discount amount = £2.40. Order Total = £21.60	Pass*
9	Pizza count just below minimum	Pizzas: 0, then 1 Pizza 1: size 1, 1 topping Delivery: false	Error, must order minimum 1 and maximum 6 pizzas, then asks again. 1 pizza accepted. Order Total = £4.00	Error shown once, then asked again. 1 pizza accepted. Discount not earned. Order Total = £4.00	Pass
10	Pizza count just above maximum	Pizzas: 7, then 1 Pizza 1: size 1, 1 topping Delivery: false	Error, must order minimum 1 and maximum 6 pizzas, then asks again. 1 pizza accepted. Order Total = £4.00	Error shown once, then asked again. 1 pizza accepted. Discount not earned. Order Total = £4.00	Pass
11	No extra toppings (0)	Pizzas: 1 Pizza 1: size 1, 0 toppings Delivery: false	Topping cost £0.00. Pizza 1 £3.25. No discount. Order Total = £3.25	0 toppings charged £0.75 (pizza cost shown as £4, expected £3.25). Discount not earned. Order Total = £4.00	Fail

12	Four extra toppings (start of '4 or more')	Pizzas: 1 Pizza 1: size 2, 4 toppings Delivery: false	Pizza 1 £8.00. No discount. Order Total = £8.00	Pizza cost shown while ordering: £8. Discount not earned. Order Total = £8.00	Pass
13	Subtotal exactly £20.00 (no discount)	Pizzas: 5 Every pizza: size 1, 1 topping Delivery: false	Each pizza £4.00. No discount. Order Total = £20.00	Pizza costs shown while ordering: £4 each. Discount not earned. Order Total = £20.00	Pass
14	Subtotal just over £20 (£20.10)	Pizzas: 3 Pizza 1: size 1, 2 toppings Pizza 2: size 2, 3 toppings Pizza 3: size 2, 4 toppings Delivery: false	Pizza 1 £4.60; Pizza 2 £7.50; Pizza 3 £8.00. Discount applied (£2.01). Order Total = £18.09	Pizza costs shown while ordering: £4.6, £7.5, £8. Discount amount = £2.01. Order Total = £18.09	Pass*
15	Delivery must not count towards the £20 limit	Pizzas: 3 Every pizza: size 2, 1 topping Delivery: true	Each pizza £6.25. No discount; delivery £2.50 added. Order Total = £21.25	Pizza costs shown while ordering: £6.25 each. Discount not earned. Order Total = £21.25	Pass
16	Pizza size just outside range	Pizzas: 1 Pizza 1: size 0, then 4, then 1; 1 topping Delivery: false	Error: size must be within range (each time), then asks again. Size 1 accepted. Order Total = £4.00	Error shown twice, then asked again. Size 1 accepted. Discount not earned. Order Total = £4.00	Pass
17	Negative number of toppings	Pizzas: 1 Pizza 1: size 1; toppings -1, then 1 Delivery: false	Error, you can't have negative toppings, then asks again. 1 topping accepted. Order Total = £4.00	Error shown, but 'Ok you've selected -1 extra toppings' is also printed. Asked again, 1 topping accepted. Order Total = £4.00	Pass

Extreme data

Test no.	Description	Input data	Expected result	Actual result	P / F

18	Pizza count far outside range	Pizzas: 100, then -5, then 1 Pizza 1: size 1, 1 topping Delivery: false	Error message each time, then asks again. 1 pizza accepted. Order Total = £4.00	Error shown twice, then asked again. 1 pizza accepted. Discount not earned. Order Total = £4.00	Pass
19	Very large number of toppings	Pizzas: 1 Pizza 1: size 1, 50 toppings Delivery: false	Topping cost capped at £2.50. Pizza 1 £5.75. No discount. Order Total = £5.75	Pizza cost shown while ordering: £5.75. Discount not earned. Order Total = £5.75	Pass
20	Pizza size far outside range	Pizzas: 1 Pizza 1: size 99, then 1; 1 topping Delivery: false	Error: size must be within range, then asks again. Size 1 accepted. Order Total = £4.00	Error shown once, then asked again. Size 1 accepted. Discount not earned. Order Total = £4.00	Pass
21	Blank customer details	Name: press Enter with nothing typed, then Sam Taylor Address: spaces only, then 1 High Street Phone: 07123456789 Pizzas: 1; size 1, 1 topping; Delivery: false	'This cannot be blank, try again!' shown for the blank name and the spaces-only address. Asks again until real details are entered. Order continues, Order Total = £4.00	'This cannot be blank, try again!' shown twice (blank name, spaces-only address). Details accepted once valid. Order continues. Order Total = £4.00	Pass
22	Largest possible order	Pizzas: 6 Every pizza: size 3, 10 toppings Delivery: true	Each pizza £9.65. Discount applied (£5.79); delivery £2.50 added. Order Total = £54.61	Pizza costs shown while ordering: £9.65 each. Discount amount = £5.79. Order Total = £54.61	Pass*
23	Text or decimal for number of pizzas	Pizzas: abc, then 2.5, then 1 Pizza 1: size 1, 1 topping Delivery: false	Inputs rejected and asked again (no crash, no calculation). 1 pizza accepted. Order Total = £4.00	Not traced: depends on how Flowgorithm handles text or a decimal typed into an Integer input. Run to record.	To run
24	Invalid delivery answer	Pizzas: 1; size 1, 1 topping Delivery: maybe, then false	'maybe' rejected and asked again (no crash). false accepted. Order Total = £4.00	Not traced: depends on how Flowgorithm handles an invalid	To run

				Boolean input. Run to record.	